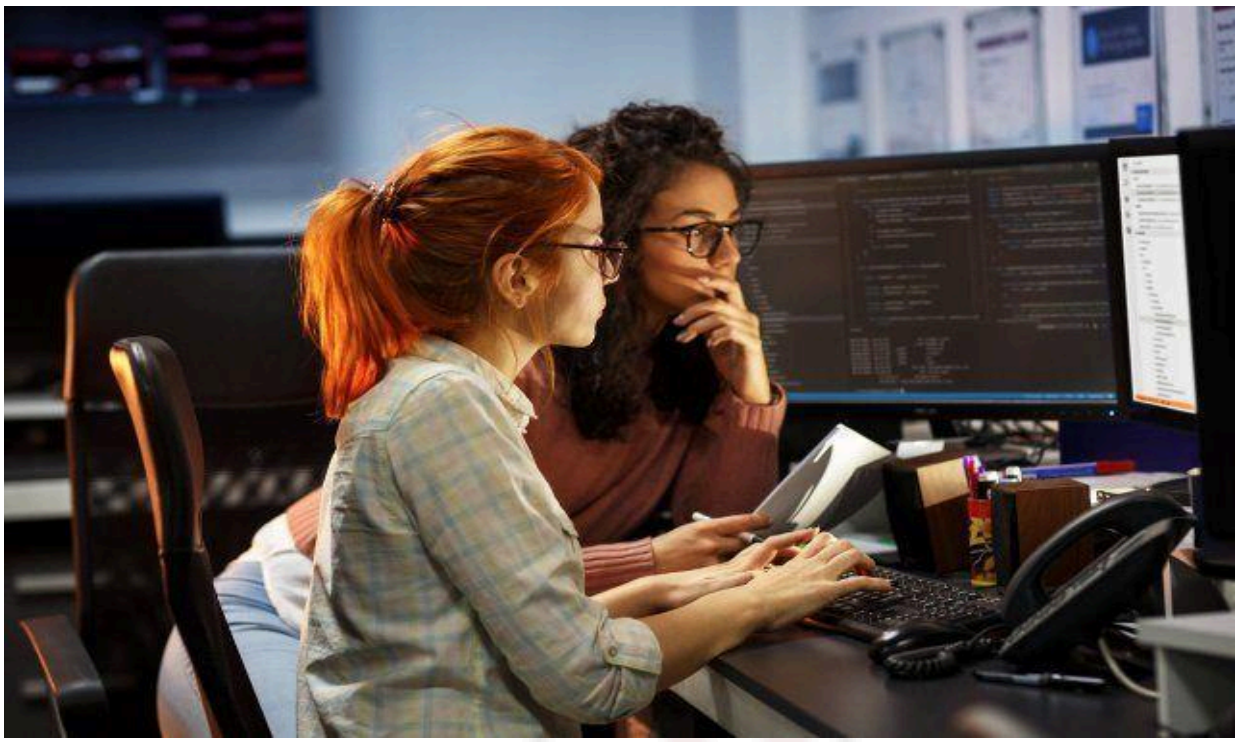


Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Programación Orientada a Objetos	Unidad #: 1	Tema: Programación y Principios de Diseño Orientado a Objetos.
	SEMANA 1	Autor:

Introducción:

Bienvenidos al material de lectura #01 de la Unidad #1 de la materia Lógica de Programación. En este documento exploramos la introducción Pensamiento Computacional y Abstracción sin antes comenzar con conocimiento de control de versiones.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en las actividades propuestas, ya que contribuirán significativamente a tu proceso de aprendizaje.

1.1. Introducción al Control de Versiones e Infraestructura de Desarrollo

Objetivo: Comprender que el software no es estático, sino un producto evolutivo que requiere gestión de cambios, historial y entornos controlados.

A. El Ecosistema del Desarrollador (The Dev Environment)

- **El Intérprete vs. El IDE:**
  - Diferencia técnica entre el lenguaje (Python Interpreter) y la herramienta de edición (VS Code).
  - Configuración del *Workspace*: Extensiones esenciales (Python, Pylance, Error Lens), Linter (flake8) y Formateadores (Black/Autopep8) para garantizar calidad desde la línea 1.
- **La Terminal (CLI - Command Line Interface):**
  - Navegación en el sistema de archivos (`cd`, `ls/dir`, `mkdir`, `touch`).
  - Por qué los ingenieros prefieren la CLI sobre la GUI (Interfaz Gráfica): Automatización, velocidad y acceso a servidores remotos.

B. Git: La Máquina del Tiempo

- **Sistemas de Control de Versiones (VCS):** Centralizados vs. Distribuidos (Git).
- **El Ciclo de Vida del Dato en Git (The Three Trees):**
  1. **Working Directory:** Donde editas los archivos (Tu mesa de trabajo).
  2. **Staging Area (Index):** Donde preparas qué cambios vas a guardar (La caja de envío).
  3. **Repository (HEAD):** Donde se guarda la "foto" permanente del proyecto (El almacén).

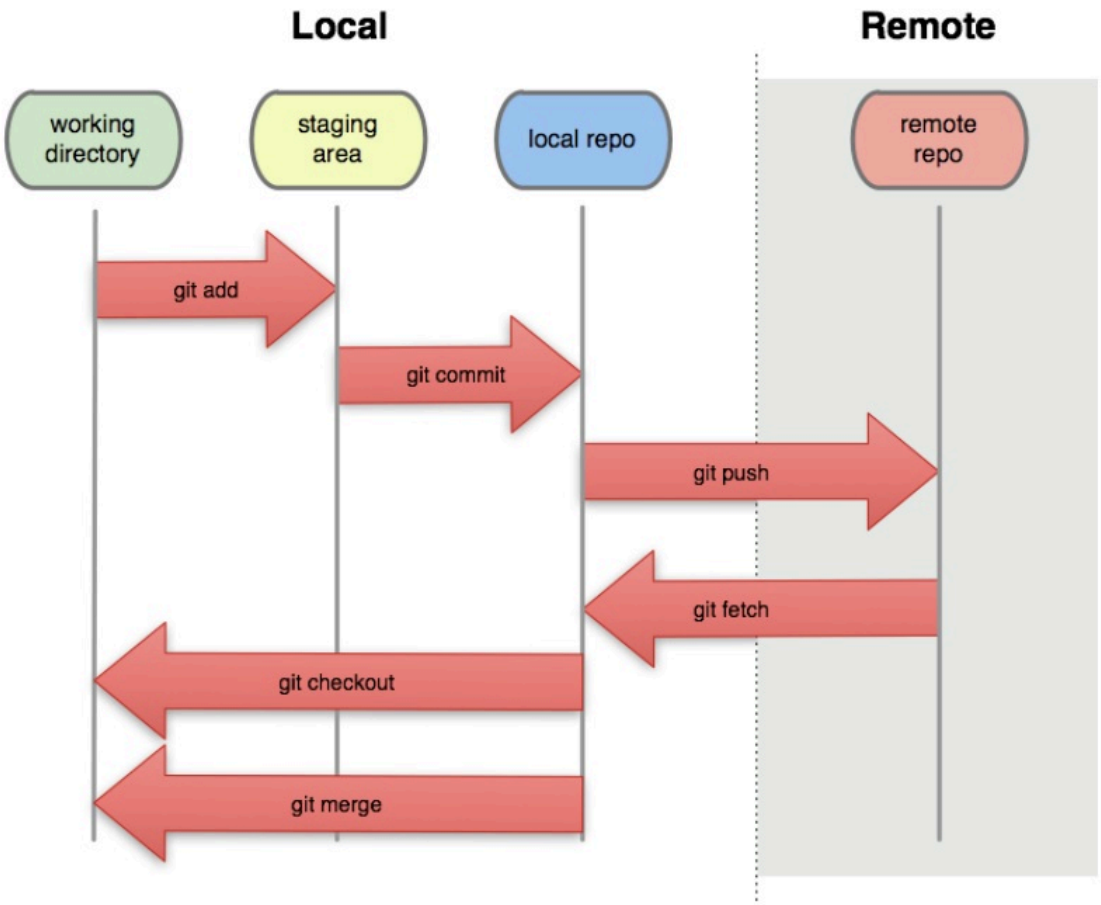
¿Te ha gustado lo que has leído hasta ahora?

Anota a continuación los aspectos que consideras más importantes de esta sección:

- **El archivo `.gitignore`:** La importancia de ignorar archivos basura, binarios compilados (`__pycache__`, `.exe`) y credenciales de seguridad.

C. Flujo de Trabajo (Workflow) Individual

- `git init`: Inicialización de un proyecto.
- `git status`: El comando más importante para saber "dónde estoy".
- `git add .` vs `git add archivo`: Granularidad en los cambios.
- `git commit -m "Verbo imperativo + Contexto"`: Buenas prácticas de mensajes de commit (Semantic Commits).
- `git log`: Auditoría y revisión histórica.



1.2. Pilares del Pensamiento Computacional: La Ingeniería Mental

Objetivo: Entrenar el cerebro para abordar problemas complejos sistemáticamente antes de escribir una sola línea de código.

A. Descomposición (Divide y Vencerás)

- **Concepto:** Romper un problema complejo en sub-problemas más pequeños, manejables y comprobables independientemente.
- **Aplicación en Ingeniería:**
  - *Problema:* "Crear un e-commerce".
  - *Descomposición:* Módulo de Usuarios, Catálogo de Productos, Carrito de Compras, Pasarela de Pago.
- **Estrategia Top-Down:** Ir de lo general a lo específico.

B. Reconocimiento de Patrones (Pattern Recognition)

- **Concepto:** Identificar similitudes o características compartidas entre problemas.
- **El Principio DRY (Don't Repeat Yourself):** Si resuelves el mismo problema dos veces, estás perdiendo tiempo.
- **Ejemplo:** Si sabes calcular el área de un cuadrado, ¿puedes generalizarlo para un rectángulo? (Patrón: Base x Altura).

C. Abstracción (El "Black Box")

- **Concepto:** Filtrar y descartar los detalles irrelevantes para centrarse en la información esencial que define el problema.
- **Modelado de Datos:** Al crear una App de Uber, ¿necesitamos saber el color de ojos del conductor? No. ¿Necesitamos su placa y ubicación? Sí. Eso es abstraer.
- **Capas de Abstracción:** Cómo el software oculta la complejidad del hardware.

D. Diseño Algorítmico

- **Definición Formal:** Un algoritmo es una secuencia finita, ordenada y no ambigua de pasos para resolver un problema.
- **Características Clave:**

- **Finito:** Debe terminar en algún momento.
- **Definido:** Cada paso tiene una sola interpretación.
- **Entrada/Salida:** Recibe datos y produce resultados.

1.3. Diseño de Algoritmos y Lógica de Procesos

Objetivo: Formalizar la solución de problemas mediante herramientas estandarizadas de representación lógica.

A. El Modelo IPO (Input - Process - Output)

- El ciclo de vida fundamental de cualquier programa:
  1. **Entrada (Input):** Datos crudos, lecturas de sensores, tipeo del usuario.
  2. **Procesamiento (Process):** Cálculos, decisiones lógicas, transformaciones.
  3. **Salida (Output):** Información útil mostrada en pantalla, guardada en archivo o enviada a la red.

B. Pseudocódigo Estructurado

- **¿Qué es?** Un lenguaje intermedio entre el lenguaje natural (español/inglés) y el lenguaje de programación.
- **Palabras Reservadas Estándar:**
  - `INICIO` / `FIN`
  - `LEER` (Read) / `ESCRIBIR` (Print)
  - `DEFINIR` (Declaración de variables y tipos)
  - `SI` ... `ENTONCES` ... `SINO` (Condicionales)
  - `MIENTRAS` / `HACER` (Ciclos)
- **Indentación:** La importancia visual de la jerarquía en el código.

C. Diagramas de Flujo (Flowcharts)

- Uso de simbología estándar ANSI/ISO 5807:
  - **Óvalo:** Inicio/Fin.
  - **Rombo:** Decisión (Bifurcación Sí/No).
  - **Paralelogramo:** Entrada/Salida de datos.
  - **Rectángulo:** Proceso/Operación aritmética.
  - **Flechas:** Flujo de control.

D. La Prueba de Escritorio (Trace Table)

- **Concepto:** Ejecución manual del algoritmo ("Jugar a ser la computadora").
- **Técnica:** Crear una tabla con una columna por cada variable y una fila por cada paso del algoritmo.
- **Objetivo:** Validar la lógica y encontrar *Bugs* lógicos antes de programar.
    - *Ejemplo:* Verificar si un bucle infinito ocurre porque la variable contador no se incrementa.